COMP102 Lecture 12: Introduction to Databases

Relational model, tables, keys, schemas, ER concepts

Data Structures with Python
June 22, 2026

UM6P University Mohammed VI Polytechnic
THE UM6P VANGUARD CENTER
#Empowering Minds.

Why Databases?

Where we are in the course

For eleven lectures we built **in-memory** data structures:

- Lists, stacks, queues, linked lists.
- Trees, heaps, hash tables, graphs.

They are fast and expressive — but they all share one assumption:

> The data lives **inside one running program**, in RAM, and disappears the moment the process ends.

Today we relax that assumption. We ask: what if the data must *outlive* the program, be *shared*, and grow *larger than memory*?

What is a database?

Before we motivate the details, let us name the thing precisely.

Definition 1.1 Database

A **database** is an organized, *persistent* collection of related data, structured so it can be efficiently stored, queried, and updated — and shared safely by many users at once.

Definition 1.2 Database Management System (DBMS)

A **DBMS** is the software that stores a database and enforces its guarantees — for example PostgreSQL, MySQL, SQLite, Oracle, or MongoDB.

We *design* the database; the DBMS does the heavy lifting. The rest of today is about **how** that data is organized — the *relational model*.

The four pressures that break in-memory structures

Our definition quietly demanded four things. Each is something a plain program in memory cannot give you:

- **Persistence.** Data must survive restarts, crashes, and power loss.
- **Sharing.** Many users and programs read and write the *same* data at the same time.
- **Scale.** The data is far larger than RAM; we cannot load it all at once.
- **Integrity.** The data must stay correct: no half-finished updates, no contradictions.

A pile of files gives you none of these four guarantees for free; a DBMS gives you all four.

“Can’t I just use a Python dictionary?”

A perfectly natural first instinct:

```
students = {
    1: {"name": "Amal",  "major": "CS"},
    2: {"name": "Yassine", "major": "Math"},
}
# ... and save it to disk?
import pickle
pickle.dump(students, open("students.pkl", "wb"))
```

This works for a toy. But ask the hard questions:

- Two programs writing at once — who wins?
- Find every CS student among 50 million rows — without loading all of them?
- A crash mid-write — is the file now corrupt?

Files and ad-hoc structures: where they stop

Table: Why hand-rolled storage does not scale

Need	Files / pickle / CSV	DBMS
Query by content	Scan everything yourself	Indexed, optimized queries
Concurrent writes	You write the locking	Transactions, isolation
Crash safety	Hope for the best	Atomic, durable commits
Integrity rules	Scattered in app code	Declared once, enforced always
Larger than RAM	Manual paging	Buffer management built in

The DBMS is not just "a place to put data" — it is a guarantee engine.

A small vocabulary before we start

- **Data model** — the abstract rules for *how* data is structured (e.g. the relational model).
- **Schema** — the structure of *a particular* database (its tables and rules).
- **Instance** — the actual data sitting in that schema *right now*.
- **DBMS** — the engine (PostgreSQL, MySQL, SQLite, MongoDB, . . .).
- **Query language** — how we ask questions (SQL — next lecture).

Remark 1.1. Analogy

The data model is the *grammar*. The schema is the *sentence structure* you chose. The instance is *today's sentence*. The DBMS is the *language itself*, enforcing the rules.

The Relational Model

Data as tables: the idea that won the database world

The big idea (Codd, 1970)

Edgar F. Codd proposed something radical: store **all** data as simple **tables**, and describe everything — including relationships — using the mathematics of **sets** and **relations**.

id	name	major
1	Amal	CS
2	Yassine	Math
3	Salma	CS

One uniform shape — the table — for everything. Simple to reason about, simple to query.

The words we will use precisely

id	name	major
1	Amal	CS
2	Yassine	Math

- **Relation** = the table itself (`Student`).
- **Tuple** = a row (one student).
- **Attribute** = a column (`name`).
- **Degree** = number of attributes (here, 3).
- **Cardinality** = number of tuples (here, 2).
- **Domain** = the allowed values of an attribute (e.g. `id` $\in \mathbb{Z}^+$).

A relation schema, written formally

Definition 2.1 Relation schema

A relation schema is a name together with a set of attributes, each with a domain:

$$R(A_1 : D_1,\ A_2 : D_2,\ \ldots,\ A_n : D_n).$$

A *tuple* is an element of the product of the domains:

$$t \in D_1 \times D_2 \times \cdots \times D_n,$$

and a *relation instance* is a finite **set** of such tuples.

- Example: $\texttt{Student}(\texttt{id} : \mathbb{Z}^+,\ \texttt{name} : \text{Text},\ \texttt{major} : \text{Text})$.
- "A relation is a set of tuples" is the single most important sentence today.

Three rules that fall out of "set of tuples"

Because a relation *is* a set:

1. **No duplicate rows.** A set has no repeated elements — every tuple is distinct.
2. **Rows have no order.** $\{a, b\} = \{b, a\}$. Sorting is a query-time choice, not a property of the table.
3. **Cells are atomic.** Each cell holds one indivisible value — no lists, no nested tables inside a cell (this is *First Normal Form*).

Remark 2.1. Connection to Python

You already met this idea: a `set` forbids duplicates and has no meaningful order. A relation is a set — of rows.

Schema vs. instance

Definition 2.2 Schema and instance

The **schema** is the *structure*: table names, attributes, domains, and rules. It changes rarely.
The **instance** is the *content*: the actual set of tuples right now. It changes constantly.

- Schema: `Student(id, name, major)` — decided once.
- Instance today: 3 rows. Instance tomorrow: 4 rows. *Same schema.*

Design the schema with care — it is the contract. The instance comes and goes.

Your turn: is this a valid relation?

id	name	phones
1	Amal	0600…, 0611…
2	Yassine	0622…
1	Amal	0600…, 0611…

Spot the two violations

Take 60 seconds. Which relational rules does this table break?

Your turn: is this a valid relation?

id	name	phones
1	Amal	0600…, 0611…
2	Yassine	0622…
1	Amal	0600…, 0611…

Spot the two violations

Take 60 seconds. Which relational rules does this table break?

- The `phones` cell holds *multiple* values — not atomic (breaks 1NF).
- Rows 1 and 3 are *identical* — duplicates are impossible in a set.

Keys & Integrity

How rows are identified and how tables connect

Why we need keys

A relation is a set of tuples — so every tuple must be **distinguishable**. Two questions drive everything about keys:

- **Identity:** given a row, what makes it *this* row and not another?
- **Reference:** how does one table point to a row in another table?

Keys are how the relational model expresses **identity** and **links**.

Superkey, candidate key, primary key

Definition 3.1 Key hierarchy

A **superkey** is a set of attributes whose values are unique across all tuples.
A **candidate key** is a *minimal* superkey — remove any attribute and uniqueness is lost.
A **primary key** is the candidate key we *choose* as the official identifier.

For `Student(id, name, email, major)`:

- $\{\texttt{id}, \texttt{name}\}$ is a superkey (unique, but not minimal).
- $\{\texttt{id}\}$ and $\{\texttt{email}\}$ are candidate keys (each minimal and unique).
- We pick $\{\texttt{id}\}$ as the **primary key**.

Choosing a primary key: natural vs. surrogate

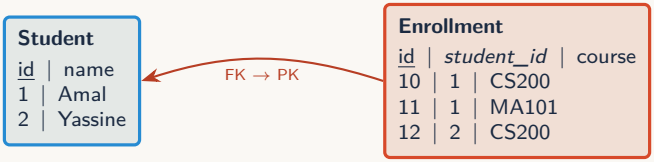
- **Natural key** — a real-world identifier (e.g. national ID, email).
  - Meaningful, but can change, be reused, or be missing.
- **Surrogate key** — a system-generated number with no meaning (e.g. auto-increment `id`).
  - Stable, compact, never changes — the common default.

Remark 3.1. Rule of thumb

Prefer a small, stable, never-reused **surrogate** primary key. Keep natural keys too, but enforce them as *unique* candidate keys rather than as the primary identifier.

Linking tables: the foreign key

We split data across tables, then connect them. A **foreign key** (FK) is an attribute in one table whose value must match a primary key in another.



`Enrollment.student_id` is a foreign key referencing `Student.id`.

Referential integrity

Definition 3.2 Referential integrity

Every foreign-key value must either match an existing primary-key value in the referenced table, or be NULL. *No dangling references are allowed.*

The DBMS enforces this for you. It will **refuse**:

- inserting an `Enrollment` for `student_id = 99` when no such student exists;
- deleting `Student` 1 while enrollments still point to it (unless you say what should happen — e.g. cascade).

Declare the rule once; the database guarantees it forever. This is integrity you cannot forget to check.

The subtle one: null

NULL is not zero and not the empty string. It marks a value that is **unknown** or **not applicable**.

- Comparisons with NULL yield *unknown*, not true/false — SQL uses **three-valued logic**.
- `salary` = NULL is never true; you must test `IS NULL`.
- A primary key may **never** be NULL (entity integrity).

Remark 3.2. Why it matters

Many real bugs come from treating NULL like an ordinary value. Decide deliberately: should this attribute ever be allowed to be unknown?

Your turn: keys check

Consider `Course`(`code`, `title`, `credits`, `department`), where each course code (e.g. `CS200`) is unique.

Answer for yourself

1. Name one candidate key.
2. Is {`title`} a good primary key? Why or why not?
3. Where would a foreign key to `Course` appear?

Your turn: keys check

Consider `Course(code, title, credits, department)`, where each course code (e.g. `CS200`) is unique.

Answer for yourself

1. Name one candidate key.
2. Is $\{$`title`$\}$ a good primary key? Why or why not?
3. Where would a foreign key to `Course` appear?

- $\{$`code`$\}$ is a natural candidate key.
- $\{$`title`$\}$ is risky — titles can repeat or change; not guaranteed unique.
- In any table recording something *about* a course (e.g. `Enrollment.course_code`).

Entity–Relationship Modeling

Modeling reality before we draw any table

Three levels of design

We do not jump straight to tables. We design in layers:

- **Conceptual** — what exists in the world and how it relates (the *ER model*). No DBMS yet.
- **Logical** — the relational schema: tables, keys, foreign keys.
- **Physical** — indexes, storage, file layout (the DBMS handles most of this).

ER modeling lets us think clearly about the *domain* first, then translate to tables mechanically.

Entities and entity sets

Definition 4.1 Entity

An **entity** is a distinguishable thing in the domain (a specific student, a specific course).
An **entity set** is a collection of entities of the same type.

- In Chen notation, an entity set is drawn as a **rectangle**.
- Entity set $\approx$ a future table. Entity $\approx$ a future row.

Attributes and their flavors

Attributes describe an entity. Several kinds appear in practice:

- **Key attribute** — uniquely identifies the entity (drawn underlined).
- **Simple** — atomic (age).
- **Composite** — decomposes into parts (`address` → street, city, zip).
- **Multivalued** — can hold several values (`phones`).
- **Derived** — computable from others (age from `birthdate`).

Remark 4.1. Foreshadowing

Composite and multivalued attributes do not fit a single atomic cell — we will *flatten* or *split* them when we move to tables.

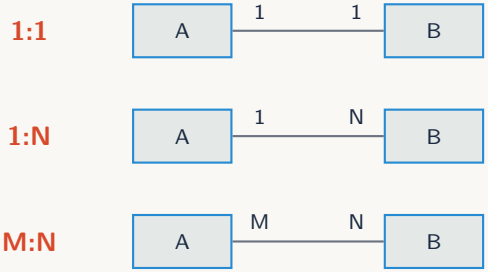
Definition 4.2 Relationship

A **relationship** associates two (or more) entities. In Chen notation it is drawn as a **diamond** between the connected entity sets.

- "A **Student** *enrolls in* a **Course**."
- Relationships can carry their own attributes (e.g. the `grade` of an enrollment).

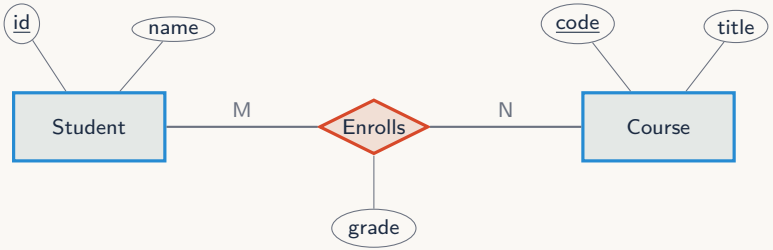
Cardinality: how many relate to how many

The **cardinality** of a relationship constrains how many entities on each side may participate.



- **1:1** — a person has one passport, a passport belongs to one person.
- **1:N** — a department has many employees; each employee is in one department.
- **M:N** — a student takes many courses; a course has many students.

A complete little ER diagram



- Rectangles = entity sets; diamond = relationship; ellipses = attributes; underline = key.
- `grade` sits on the *relationship* — it belongs to neither student nor course alone.

Your turn: model it

A library lends books to members.

Sketch the ER model

1. What are the entity sets?
2. What relationship connects them, and what is its cardinality?
3. Where does the `due_date` of a loan belong?

Your turn: model it

A library lends books to members.

Sketch the ER model

1. What are the entity sets?
2. What relationship connects them, and what is its cardinality?
3. Where does the `due_date` of a loan belong?

- Entities: **Member**, **Book** (and possibly **Copy** of a book).
- Relationship: **Borrows**, cardinality **M:N** over time (a member borrows many books; a book is borrowed by many members).
- `due_date` is an attribute *of the relationship*, not of member or book.

From ER to Tables

Translating a conceptual model into a relational schema

The mapping recipe

Turning an ER diagram into tables is mostly mechanical. Four rules cover the common cases:

1. **Entity set** $\rightarrow$ a table; its key attribute $\rightarrow$ the primary key.
2. **1:N relationship** $\rightarrow$ a foreign key on the "many" side.
3. **M:N relationship** $\rightarrow$ a new *junction* table with two foreign keys.
4. **Multivalued attribute** $\rightarrow$ a separate table linked back by a foreign key.

Good ER modeling makes this step almost automatic.

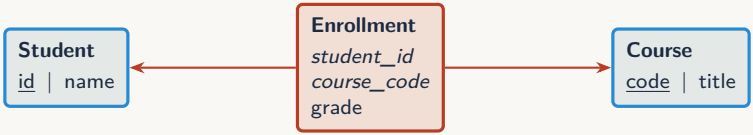
Rule 2 in pictures: 1:N becomes a foreign key

"A **Department** has many **Employees**." The foreign key goes on the *many* side.

Each employee row stores the `dept_id` of its one department. Many employees can share the same value — that is the "many".

Rule 3: M:N needs a junction table

You *cannot* represent M:N with a single foreign key — a cell is atomic, so a student row cannot list many courses.



- `Enrollment` holds one row per (student, course) pair.
- Its primary key is the *composite* {`student_id`, `course_code`}; both are also foreign keys.
- Relationship attributes (`grade`) live here.

Worked example: the full university schema

Mapping our ER diagram (Student — Enrolls — Course) gives **three** tables:

Table: University schema (logical design)

Table	Attributes	Keys
Student	`id`, name, major	PK: `id`
Course	`code`, title, credits	PK: `code`
Enrollment	`student_id`, `course_code`, grade	PK: (`student_id`, `course_code`); FKs to Student, Course

The M:N "Enrolls" relationship became its own table. This is the single most common pattern in real schemas.

A glimpse of the physical layer (next lecture)

Declaring this schema in SQL — a preview of Lecture 13:

```sql
CREATE TABLE Student (
    id      INTEGER PRIMARY KEY,
    name    TEXT NOT NULL,
    major   TEXT
);

CREATE TABLE Enrollment (
    student_id  INTEGER REFERENCES Student(id),
    course_code TEXT    REFERENCES Course(code),
    grade       TEXT,
    PRIMARY KEY (student_id, course_code)
);
```

Notice: keys and referential integrity are *declared*, not coded. The DBMS enforces them.

Designing Good Schemas

Why one fat table is a trap, and how to avoid it

The temptation: keep everything in one table

Why not store enrollments, student info, and course info together?

sid	sname	course	ctitle	grade
1	Amal	CS200	Algorithms	A
1	Amal	MA101	Calculus	B
2	Yassine	CS200	Algorithms	A

Looks convenient — but "Amal" and "Algorithms" are now stored **repeatedly**. Redundancy is the root of trouble.

Redundancy breeds three anomalies

- **Update anomaly.** Amal changes her name $\Rightarrow$ you must fix *every* row, or the data contradicts itself.
- **Insertion anomaly.** You cannot record a new course with no students yet — there is no row to put it in.
- **Deletion anomaly.** Delete the last enrollment in `CS200` and you accidentally erase the fact that `CS200` exists.

Each independent fact should be stored in exactly **one** place. Redundancy is not just wasteful — it makes correctness impossible to maintain.

Normalization, intuitively

Normalization is the discipline of splitting tables so that each fact lives once. The first three normal forms, in plain words:

- **1NF** — every cell is atomic; no repeating groups or lists in a cell.
- **2NF** — (with a composite key) no attribute depends on only *part* of the key.
- **3NF** — no non-key attribute depends on another non-key attribute.

Remark 6.1. The slogan

"Each non-key attribute depends on the key, the whole key, and nothing but the key."

The fix: decompose into clean tables

The fat table normalizes back into exactly the schema we designed via ER:

Good ER modeling and normalization *converge* on the same answer. Two roads, one well-designed schema.

When *not* to fully normalize

Normalization is the default, not a religion. Sometimes we deliberately keep redundancy:

- **Read performance.** Joining many tables can be slow; controlled duplication (*denormalization*) can speed up heavy read workloads.
- **Analytics / warehouses.** Reporting systems often favor wide, redundant tables.
- **NoSQL document stores.** Embedding related data in one document is sometimes the right call (Lecture 14).

Remark 6.2. Engineering judgment

Normalize first for correctness; denormalize later, deliberately, for measured performance — never by accident.

Recap & What's Next

The arc of today

- In-memory structures vanish; **databases** give persistence, sharing, scale, integrity.
- The **relational model**: all data as tables; a relation is a *set of tuples*.
- **Keys** express identity (primary) and links (foreign); the DBMS enforces **referential integrity**.
- **ER modeling** captures the domain; cardinality (1:1, 1:N, M:N) drives the design.
- **Mapping** ER to tables: entities $\rightarrow$ tables, 1:N $\rightarrow$ FK, M:N $\rightarrow$ junction table.
- **Normalization** removes redundancy so each fact lives in one place.

Common misconceptions to leave behind

- "Rows have an order." — They do not; ordering is a query choice.
- "A primary key must be meaningful." — A surrogate key is often better.
- "NULL equals zero or empty." — It means *unknown*; it breaks ordinary comparisons.
- "M:N can be a single foreign key." — It needs a junction table.
- "More tables = worse design." — Usually the opposite: separation removes anomalies.

Next lecture: SQL Foundations

We have a well-designed schema. Next we learn to **talk** to it:

- `SELECT ... FROM ... WHERE` — asking questions.
- `JOIN` — recombining the tables we so carefully separated today.
- `GROUP BY`, `ORDER BY` — aggregating and sorting.
- Indexing basics — why queries are fast.

Today you learned to **structure** data. Next, you learn to **query** it.

Exit ticket

Before you leave, answer on paper (one line each):

Three questions

1. Why is “a relation is a set of tuples” such an important sentence?
2. Given “a project has many tasks; a task belongs to one project,” where does the foreign key go?
3. Give one real example of an M:N relationship from your own life.

Questions? Let’s discuss.